

with the training examples. The loss function tailored for ANDRE is defined as:

$$\mathcal{L}_{\text{BCE}} = \sum_{(e,h) \in \mathcal{D}} -h \cdot \log p(\hat{h} | e, \mathcal{W}) - (1-h) \cdot \log (1 - p(\hat{h} | e, \mathcal{W})), \quad (10)$$

where $p(\hat{h} | e, \mathcal{W})$ denotes the predicted probability of the head predicate given input example e and weight configuration $\mathcal{W}$, and the Boolean output $h \in \{0, 1\}$ is the corresponding ground-truth label.

Entropy Loss: To encourage exploration and prevent premature convergence to overly confident or arbitrary predicate selections, we introduce an entropy-based regularization term. For each trainable probability distribution P_{ij} associated with S_j , the total entropy loss across all probabilities in all subrules is defined as:

$$\mathcal{L}_{\text{E}} = - \sum_{i=1}^n \sum_{j=1}^m \sum_{k=1}^3 p_{ij}^{(k)} \cdot \log (p_{ij}^{(k)} + \epsilon), \quad (11)$$

where ϵ is a small constant (e.g., 10^{-6}) for numerical stability. By minimizing this loss, ANDRE encourages one of the probabilities in P_{ij} to converge to 1, while the remaining two converge to 0.

Similarity Loss: To ensure subrule diversity, we minimize the cosine similarity of each subrule weights with the rest of subrule weights, defined as:

$$\mathcal{L}_{\text{S}} = \frac{2}{n(n+1)} \sum_{i=1}^n \sum_{j=i+1}^n \frac{\langle W_i, W_j \rangle}{\|W_i\| \cdot \|W_j\|}, \quad (12)$$

where W_i and W_j are vectorized weights of subrule i and j , respectively.

In general, by minimizing $\mathcal{L}_{\text{BCE}}$, $\mathcal{L}_{\text{E}}$, and $\mathcal{L}_{\text{S}}$, ANDRE iteratively updates its parameters to emphasize the most informative subpredicates and suppress irrelevant or redundant ones. This semantic optimization enables the model to learn interpretable and high-performing logical rules.

Syntactic Learning. In addition to achieving high predictive accuracy, it is essential that the extracted rules conform to valid logical structures. Two common formats are considered to ensure syntactic validity: (1) the *head variable inclusion* format, and (2) the *auxiliary variable connectivity* format (see Appendix C for more details). To enforce these structural constraints during training, we introduce two syntactic loss functions: the *Range-Restricted Loss* $\mathcal{L}_{\text{R}}$, corresponding to head variable inclusion, and the *Connected Loss* $\mathcal{L}_{\text{C}}$, corresponding to auxiliary variable connectivity.

To compute $\mathcal{L}_{\text{R}}$ and $\mathcal{L}_{\text{C}}$, we first quantify how frequently each variable X_k appears in the body of a subrule $\hat{h}_i$, given the current weights $\mathcal{W}$. This frequency is denoted by M_k^i , representing the expected count of variable X_k in the body predicates of subrule $\hat{h}_i$. The computation begins by identifying which body predicates contain X_k as an argument. For this, we define a Boolean indicator $V_k^{ij} = \mathbb{1}(x = X_k, b = b_j)$, which evaluates to 1 if variable X_k appears in predicate B_{ij} , and 0 otherwise. This information is domain-specific and assumed to be fixed for all subrules. We then estimate the probability that each predicate B_{ij} is active in subrule $\hat{h}_i$. Since $p_{ij}^{(3)}$ represents the probability of selecting the identity subpredicate 1, the effective inclusion probability of predicate B_{ij} is $1 - p_{ij}^{(3)}$. The expected usage count of variable X_k in $\hat{h}_i$ is then computed as:

$$M_k^i = \sum_{j=1}^m (1 - p_{ij}^{(3)}) \cdot \mathbb{1}(x = X_k, b = b_j). \quad (13)$$

Range-Restricted Loss: To enforce the head variable inclusion constraint, we penalize subrules where a head variable is missing from the body. Let K_h denote the set of head variables. The loss is formulated as:

$$\mathcal{L}_{\text{R}} = \sum_{i=1}^n \sum_{k \in K_h} \begin{cases} (M_k^i - 1)^2, & \text{if } M_k^i < 1, \\ \eta (M_k^i - 1), & \text{if } M_k^i \geq 1, \end{cases} \quad (14)$$

which ensures the inclusion of head variables at least once in the body predicates. The loss increases when $M_k^i < 1$, encouraging inclusion of all head variables. Parameter $\eta \in [0, \infty)$ adjusts the magnitude of penalty when $M_k^i \geq 1$. Higher η values encourage the head variable to appear in body predicates only once.

Connected Loss: To enforce auxiliary variable connectivity, we penalize subrules where an auxiliary variable appears only once and encourage subrules to have two auxiliary variables. Let K_a denote the set of auxiliary variables. The corresponding loss is defined as:

$$\mathcal{L}_C = \sum_{i=1}^n \sum_{k \in K_a} \begin{cases} c_1 \cdot \exp(-c_2(M_k^i - 1)^2), & \text{if } M_k^i < 2, \\ (M_k^i - 2)^2, & \text{if } M_k^i \geq 2, \end{cases} \quad (15)$$

where c_1 and c_2 are tuning parameters. The bell-shaped penalty is maximized when $M_k^i \approx 1$, disfavoring singleton auxiliary variables.

Digitization Loss: Optionally, to encourage M_k^i to be an integer (e.g., 0, 1, or 2), we introduce a regularization term:

$$\mathcal{L}_D = \frac{1}{|K|} \sum_{k \in K} \frac{1}{2} \left(1 - \sin \left(2\pi M_k^i + \frac{\pi}{2} \right) \right), \quad (16)$$

where K is the set of all variable indices. While this periodic and non-convex loss can improve interpretability, it may introduce instability if over-weighted. Therefore, it should be used with a small coefficient.

Combining the semantic and syntactic objectives, the total loss is formulated as:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + \lambda_E \mathcal{L}_E + \lambda_S \mathcal{L}_S + \lambda_R \mathcal{L}_R + \lambda_C \mathcal{L}_C + \lambda_D \mathcal{L}_D, \quad (17)$$

where λ_E , λ_S , λ_R , λ_C , and λ_D are scalar weights controlling the influence of each regularization term. As detailed in Appendix D, the loss weights are scheduled during training based on the role of each loss component. The loss scheduling technique gradually introduces or reweights loss components during training, allowing the model to first learn stable, high-confidence structures before enforcing stricter semantic and syntactic constraints. By dynamically adjusting loss weights over training epochs, it prevents early over-regularization and promotes more robust, interpretable convergence. Given the total loss, optimization proceeds via gradient-based methods on a given batch of data. During training, the network minimizes $\mathcal{L}$ through backpropagation, progressively refining its weights to discover valid subrules that collectively define the same-head logical rule.

ANDRE assumes no knowledge of the number of rules in the target dataset, thereby running iteratively with different number of rules to find the most accurate model, even though it is capable of finding rules with only one-shot training. The accuracy metric used for evaluation is defined as the proportion of correctly predicted labels over all examples. At each stage, multiple restarts are performed to avoid poor local minima, ensuring that the most effective subrule is selected. This incremental process not only enhances the model convergence but also enables subrules to be naturally ranked by their learned confidence. After training, it is essential to extract the effective body predicates from the probabilities learned by ANDRE. Appendix E describes the criterion we use to identify all body predicates.

4 EXPERIMENTAL RESULTS

This section presents a comprehensive evaluation of ANDRE’s performance in learning interpretable logical rules. All experiments were implemented using PyTorch and ADAM optimizer on an NVIDIA GeForce RTX 5090. The default hyperparameter settings are provided in Table 5 (Appendix F) but may vary slightly per dataset. To ensure stable optimization, batch training with label balancing, gradient clipping, and learning rate decay were applied across all training phases. Moreover, ANDRE model is completely vectorized and GPU-friendly, having the ability to train on large-scale datasets rapidly.

Prior to the core experiments, two illustrative case studies were conducted on synthetic datasets in Appendix G to visualize how ANDRE updates its predicate probabilities, how loss and accuracy evolve over training, and how symbolic rules are extracted. For `grandparent` task, figure 4 illustrates the final learned subpredicate weights. Accordingly, after applying the predicate identification

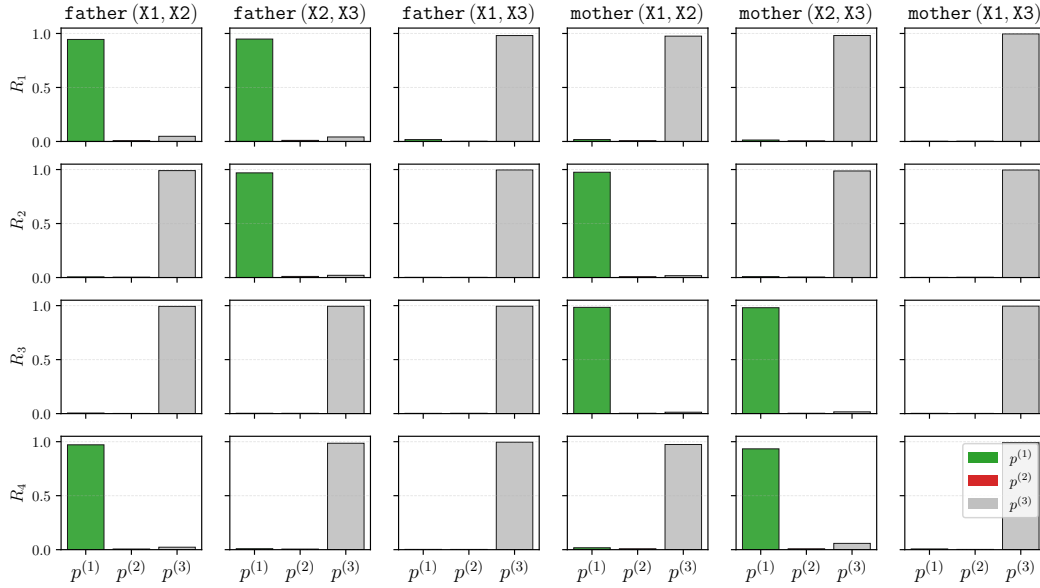


Figure 4: Final softmax-normalized subpredicate probabilities for the Grandparent task. Each subrule converges to a sparse and interpretable predicate configuration. Green, red, and gray bars indicate the probability of inclusion, negation, or exclusion of each body predicate in each subrule R_i .

procedure described in Appendix E, ANDRE extracts the following symbolic rule set:

$$\begin{aligned}
 \text{grandparent}(X_1, X_3) &\leftarrow \text{mother}(X_1, X_2) \wedge \text{mother}(X_2, X_3), \\
 \text{grandparent}(X_1, X_3) &\leftarrow \text{father}(X_1, X_2) \wedge \text{father}(X_2, X_3), \\
 \text{grandparent}(X_1, X_3) &\leftarrow \text{mother}(X_1, X_2) \wedge \text{father}(X_2, X_3), \\
 \text{grandparent}(X_1, X_3) &\leftarrow \text{father}(X_1, X_2) \wedge \text{mother}(X_2, X_3),
 \end{aligned}$$

which are both semantically and syntactically correct. Variables X_1, X_3 are head variables, while X_2 is an auxiliary variable. More details are deferred to Appendix G.

The experiments are organized into three main categories:

4.1 CLASSICAL ILP DATASETS

To assess the generalizability and interpretability of ANDRE in real-world symbolic settings, we first evaluate its performance on classical ILP benchmarks. Similar to prior methods such as ∂ ILP (Evans & Grefenstette, 2018), NeuralLP (Yang et al., 2017), and DFORL (Gao et al., 2024), ANDRE is tested on over ten widely used ILP tasks. As these benchmarks are originally unstructured, a preprocessing step is necessary to transform them into a form compatible with differentiable learning. Therefore, we apply a propositionalization procedure introduced by Gao et al. (2022), that converts the logical relationships into a supervised dataset E , consisting of m predicate valuations and a corresponding Boolean target output.

Following this conversion, ANDRE is tasked with discovering a logical rule that best explains each dataset while preserving accuracy over data. Table 1 summarizes the tasks and the performance of ANDRE compared to the baselines. Each dataset is described in full detail in Appendix H, including body predicates, background knowledge, and sets of positive and negative examples. ANDRE achieves 100% rule extraction accuracy

Table 1: The results on ILP datasets. The symbols $\checkmark$, $*$, and $-$ indicate that the accuracy of the generated logic program is equal to 100%, less than 100%, and equal to 0%, respectively (Gao et al., 2024).

Task	∂ ILP	NeuralLP	DFORL	ANDRE
Predecessor	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Odd	$\checkmark$	$-$	$\checkmark$	$\checkmark$
Even	$-$	$-$	$\checkmark$	$\checkmark$
LessThan	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Son	$\checkmark$	$*$	$\checkmark$	$\checkmark$
Grandparent	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Related	$\checkmark$	$*$	$\checkmark$	$\checkmark$
Father	$-$	$-$	$\checkmark$	$\checkmark$
Direct Edge	$\checkmark$	$*$	$\checkmark$	$\checkmark$
Connected	$\checkmark$	$*$	$\checkmark$	$\checkmark$